Retries and Delayed Messages

try again later, without blocking everything

Watch the 4-min video

In one line: transient failures deserve retries with exponential backoff and jitter, but sleeping inside a consumer blocks its partition, so move failed messages to delayed retry queues or topics, cap the attempts, and send what still fails to a DLQ.

1 Don't sleep in the consumer

A webhook call to a partner fails because their server is down. You want to try again in five minutes. In a Kafka consumer, the obvious move is to sleep and retry. But while it sleeps, every other message in that partition waits, the consumer may miss `max.poll.interval.ms` and trigger a rebalance, and one partner's outage delays webhooks for everyone.

2 Transient vs permanent

Retry **transient** failures: timeouts, 503s, connection resets, rate limits (respect `Retry-After`). Don't retry **permanent** ones: 400 Bad Request, validation errors, 'customer not found'. Those go straight to a DLQ (lesson 31). Retrying the wrong kind wastes capacity and hides bugs.

3 Backoff and jitter

Wait longer after each failure: **exponential backoff** (1 s, 2 s, 4 s, 8 s...) with a cap, so a struggling service gets room to recover. Add **jitter** (randomness) so thousands of clients don't retry at the same instant and knock it over again. Cap the total number of attempts or the total time.

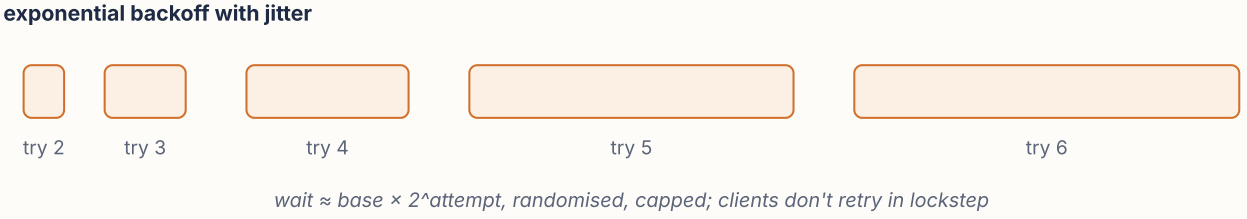


Figure 1. Growing, randomised waits spread retries out.

4 Delayed messages

For queues, use the broker's delay feature. For streams, use **retry topics** with fixed delays: the main consumer publishes a failed message to `retry-5m` with a 'not before' timestamp and moves on. The retry consumer pauses its partition until the earliest message is due, then reprocesses it. Order per key is lost for retried messages, so handlers should use versions or tolerate it (lesson 29).



Figure 2. Built-in delay mechanisms by broker.

WORKED EXAMPLE: RETRY-5M AND RETRY-1H FOR FAILED WEBHOOKS

The webhook consumer reads the `webhooks` topic and calls the partner. If the call fails with a 503, it publishes the message to `retry-5m` with headers `attempt=1` and `due=now+5m`, commits, and continues with other partners' webhooks. The retry-5m consumer sleeps until the message is due and retries; on failure it forwards to `retry-1h`; after that, to `webhooks-dlq` with an alert. A partner outage now delays only that partner's webhooks, and the partner receives the same idempotency key on every attempt.

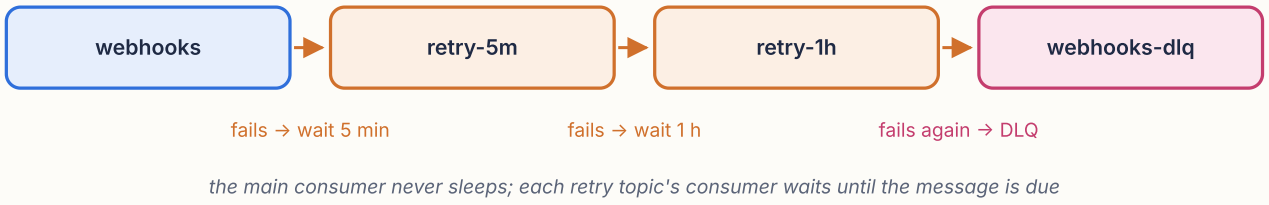


Figure 3. A retry ladder of topics with growing delays, ending in a DLQ.

COMMON MISTAKES

- Sleeping in the poll loop.
- Retrying immediately and forever, without backoff or a cap.
- Retrying non-idempotent calls without an idempotency key.
- Retrying permanent errors.

5 Key terms

Transient failure: likely to succeed if retried later

Jitter: randomness added to waits

Retry budget: limit on retries as a share of traffic

Exponential backoff: waits double after each failure

Retry topic: topic holding messages until their next attempt

Retry-After: HTTP header saying when to try again

6 Interview questions

Q1. Why use exponential backoff with jitter? BASIC

Backoff gives a failing service time to recover; jitter prevents many clients from retrying at the same moment and creating load spikes.

Q2. Why shouldn't a Kafka consumer sleep before retrying? BASIC

It blocks every message behind it in the partition and can exceed max.poll.interval.ms, triggering a rebalance.

Q3. Design retries for webhook delivery to thousands of partners. SCENARIO
Classify errors, send transient failures to retry topics with growing delays, cap attempts, DLQ the rest, use idempotency keys, and isolate partners so one outage doesn't delay others.

Q4. A downstream outage made your retries multiply the load ten times. Fix it. SCENARIO
Add backoff with jitter, a retry budget, a circuit breaker that stops calls during the outage, and a single retry layer instead of retries at every level.

Q5. What do retry topics do to ordering? TRICKY
A retried message is processed after later messages for the same key; if order matters, use versions or block the key until its retry succeeds.

Q6. How long should you keep retrying? TRICKY
As long as the business value lasts and the dependency's typical outage takes: minutes for user-facing actions, hours or days for webhooks, then DLQ and alert.

Next: SD-I-011 · Backpressure in Messaging backendroadmap.com